Pattern observer



Design patterns et Gang of Four

Qu’est ce qu’un Design patterns ?

- Une solution éprouvée à un problème de conception récurrent
- Un vocabulaire commun entre développeurs
- Un plan d’organisation de classes et d’objets
- Un code plus flexible et maintenable

Qu’est ce que le Le Gang of Four (GoF) ?

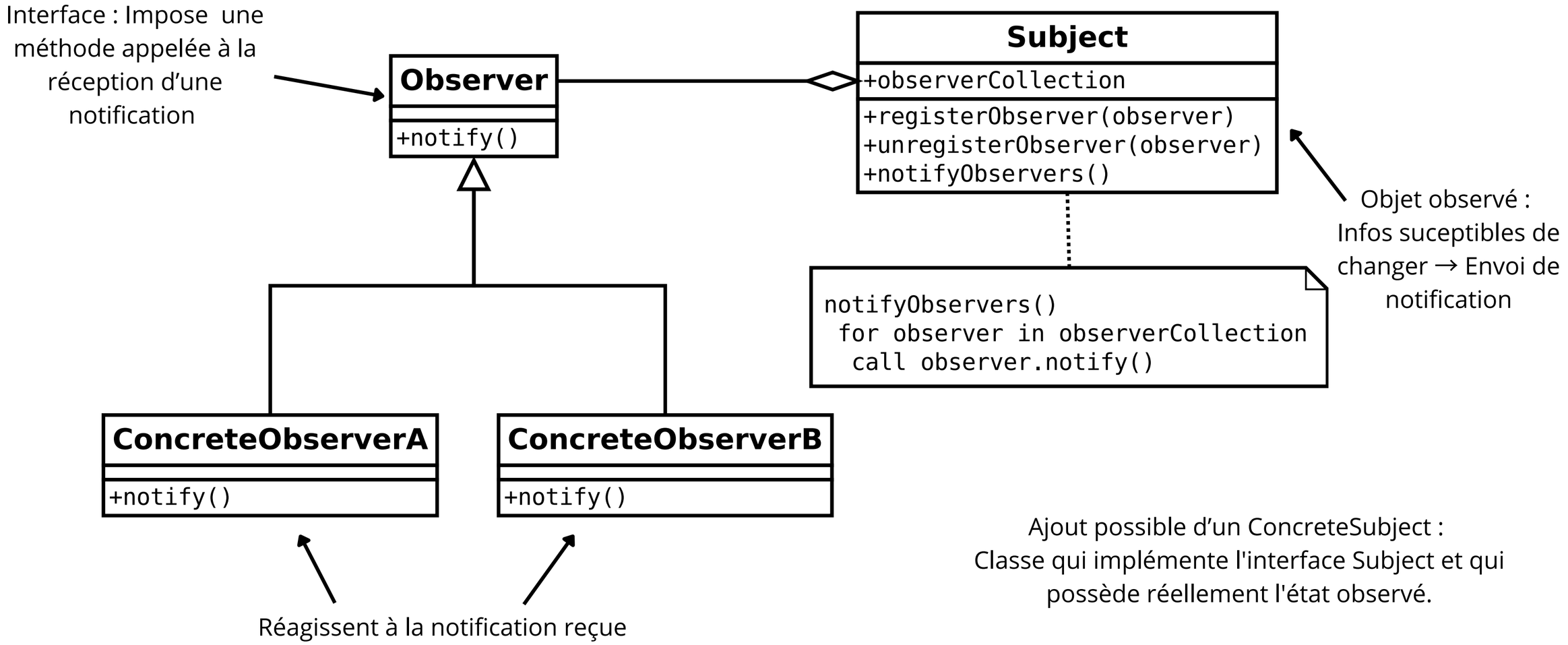
- Quatre auteurs : Erich Gamma, Richard Helm, Ralph Johnson et John Vlissides
- Livre publié en 1994 : Design Patterns: Elements of Reusable Object-Oriented Software

- Il recense 23 patterns en 3 familles :

créationnels	structuraux	comportementaux
comment créer les objets	comment les assembler	comment ils communiquent
Singleton, Fabrique…	*Adaptateur, Décorateur…*	*Stratégie,* ***Observateur****…*

Pattern Observateur : Le Diagramme Générique

Type : Comportemental



Problématique

Comment permettre à plusieurs affichages de recevoir automatiquement les changements de température d'une station météo, sans créer de dépendances directes entre celle-ci et chaque affichage ?

1. Couplage fort ≠ DIP

- AffichageTemperature
- AffichageStatistique
- AffichageHistorique

2. Non respect de OCP

Ajout d'un affichage

3. Maintenance compliquée

- Modification d'un nom de méthode
- Rétro-conception plus compliquée

```
public class StationMeteo {

    private AffichageTemperature affichage1;
    private AffichageStatistiques affichage2;
    private AffichageHistorique affichage3;

    public void nouvelleTemperature(double temperature) {
        affichage1.actualiser(temperature);
        affichage2.actualiser(temperature);
        affichage3.actualiser(temperature);
    }
}
```

Intention du pattern Observer

Prévenir d'autres objets d'un changement d'état sans avoir besoin de connaître leur fonctionnement interne

- Station météo possède une liste d'observateurs
- Changement de température → MAJ de son état
- Notifie les observateurs
- Chaque observateur réagit à sa manière

- **Faible couplage :** Respect de DIP
- **Flexibilité :** Respect de OCP
- **Maintenabilité :** Code plus claire

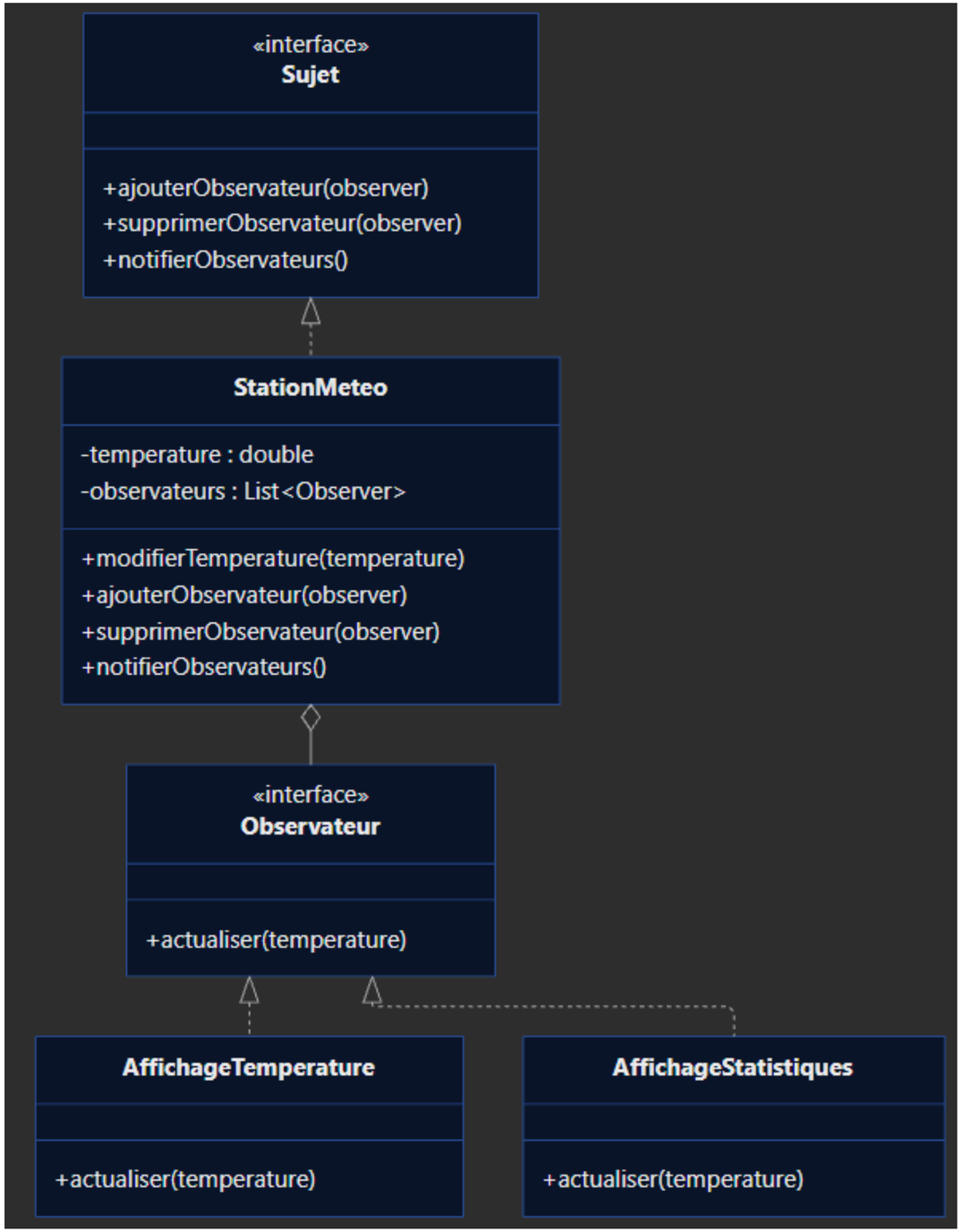
```java
public class StationMeteo {

    private List<Observer> observateurs = new ArrayList<>();

    public void nouvelleTemperature(double temperature) {
        for (Observer observateur : observateurs) {
            observateur.update(temperature);
        }
    }
}
```

ConcreteSubject →

ConcreteObserver →



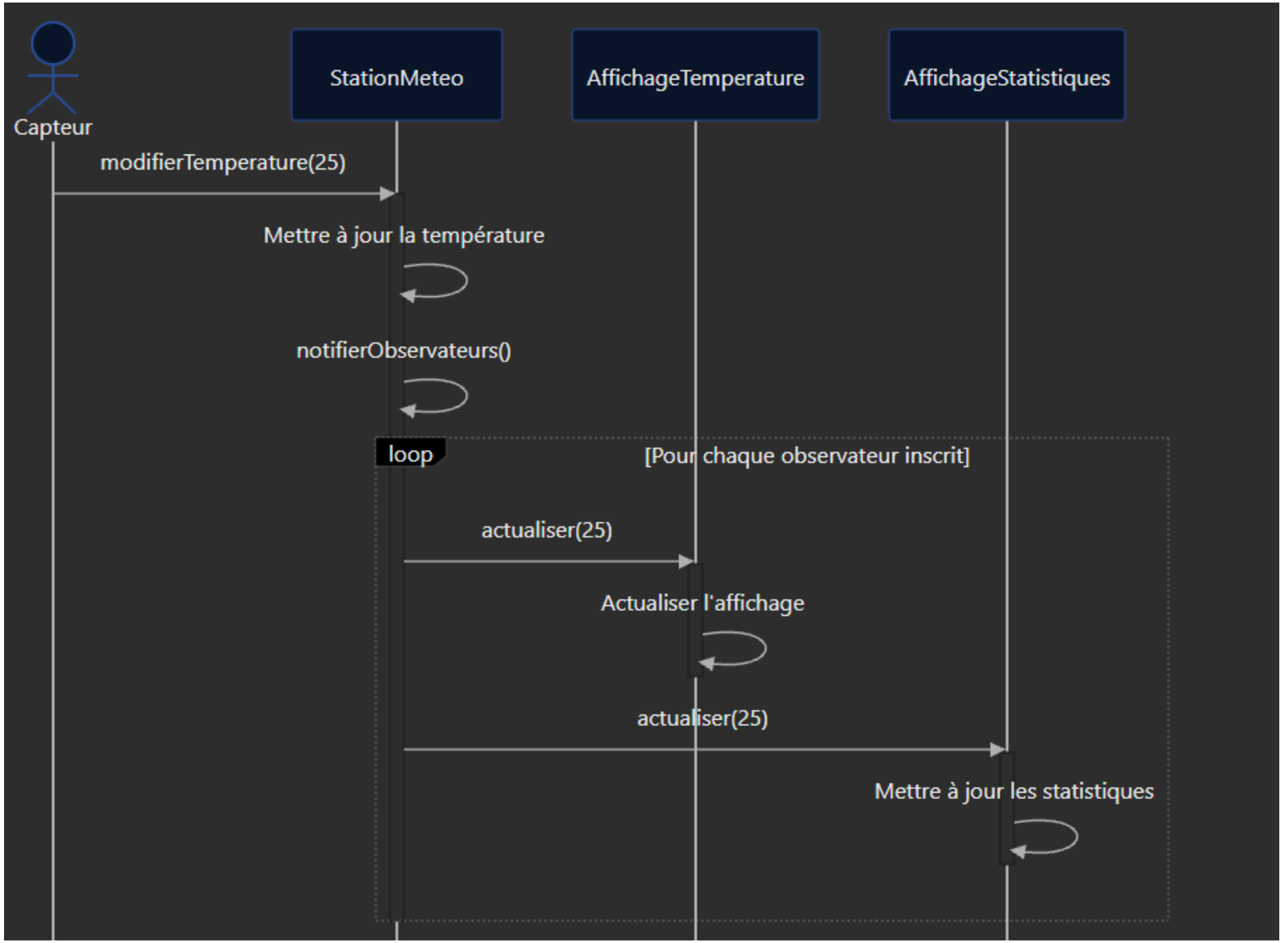

Capteur
StationMeteo
AffichageTemperature
AffichageStatistiques
modifierTemperature(25)
Mettre à jour la température
notifierObservateurs()
loop
[Pour chaque observateur inscrit]
actualiser(25)
Actualiser l'affichage
actualiser(25)
Mettre à jour les statistiques

Tableau des rôles

Role GoF	Fonction
Subject	Définit les méthodes Inscrire, Désinscrire et Notifier
ConcreteSubject	Implémente Subject, possède l’état observé et la liste de observateurs
Observer	Définit ce que chaque observateur doit pouvoir réaliser lorsqu’il est notifié
ConcreteObserver	Implémente Observer et définit sa sa réaction aux notifications

Principe	Avant	Après (avec Observer)
SRP	**StationMeteo** gère les mesures et sait comment mettre à jour chaque affichage	**StationMeteo** gère l'état et la liste des observateurs; chaque affichage gère sa propre présentation
OCP	Ajouter un affichage oblige à modifier **StationMeteo** (nouvel attribut et nouvel appel)	Ajouter **AffichageHistorique** = nouvelle classe + **ajouterObservateur**, sans toucher à **StationMeteo**
LSP	Rien à substituer, tout est en dur	N'importe quel observateur peut remplacer un autre sans rien casser
ISP	Pas d'interface	L'interface observateur ne déclare qu'une méthode, **actualiser(temperature)**
DIP	La station dépend des classes concrètes (**AffichageTemperature,** ...)	La station ne connaît que l'interface Observateur

Les Limites du Pattern Observer

- Fuite mémoire : un observateur jamais désinscrit reste en mémoire.
- Ordre de notification : il n’est pas garanti.
- Notifications en cascade : risque de boucle infinie.
- Exception dans un observateur : les suivants ne sont pas prévenus.
- Multithread : désinscription pendant une notification = erreur.
- Mises à jour inutiles : des observateurs reçoivent des données qui ne les concernent pas.

Version Push

```java
public interface Observateur {
    void actualiser(double temperature);
}
```

```java
public void notifierObservateurs() {
    for (Observateur observateur : observateurs) {
        observateur.actualiser(temperature);
    }
}
```

Version Pull

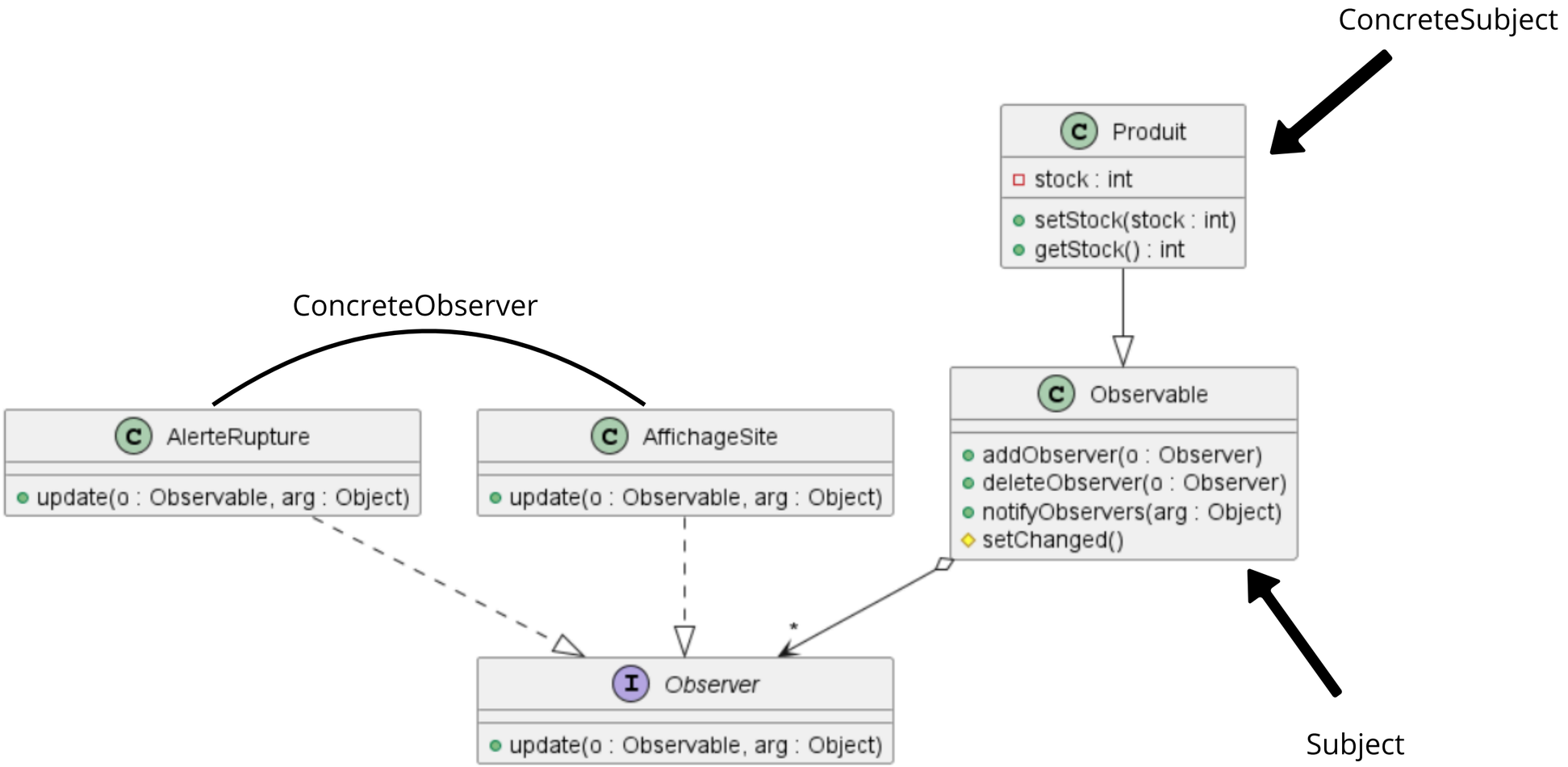
```java
public interface Observateur {
    void actualiser(StationMeteo station);
}
```

```java
public void notifierObservateurs() {
    for (Observateur observateur : observateurs) {
        observateur.actualiser(this);
    }
}

public double getTemperature() {
    return temperature;
}

public double getHumidite() {
    return humidite;
}
```

	Push	Pull
Avantages	• Simple à mettre en place • L’observateur ne connaît pas le sujet (faible couplage) • Un seul appel, pas d’aller-retour	• Chaque observateur lit seulement ce dont il a besoin • La signature de actualiser ne change pas quand on ajoute une donnée
Inconvénients	• Données envoyées même si inutiles • Nouvelle donnée = signature de actualiser à modifier partout	• L’observateur dépend du sujet • Plus d’appels • L’état peut changer entre la notification et la lecture


ConcreteSubject
Produit
stock : int
setStock(stock : int)
getStock() : int
ConcreteObserver
Observable
addObserver(o : Observer)
deleteObserver(o : Observer)
notifyObservers(arg : Object)
setChanged()
AlerteRupture
update(o : Observable, arg : Object)
AffichageSite
update(o : Observable, arg : Object)
*
Observer
update(o : Observable, arg : Object)
Subject

```java
public class Produit extends Observable {
    private int stock;

    public void setStock(int stock) {
        this.stock = stock;
        setChanged();
        notifyObservers(stock);
    }

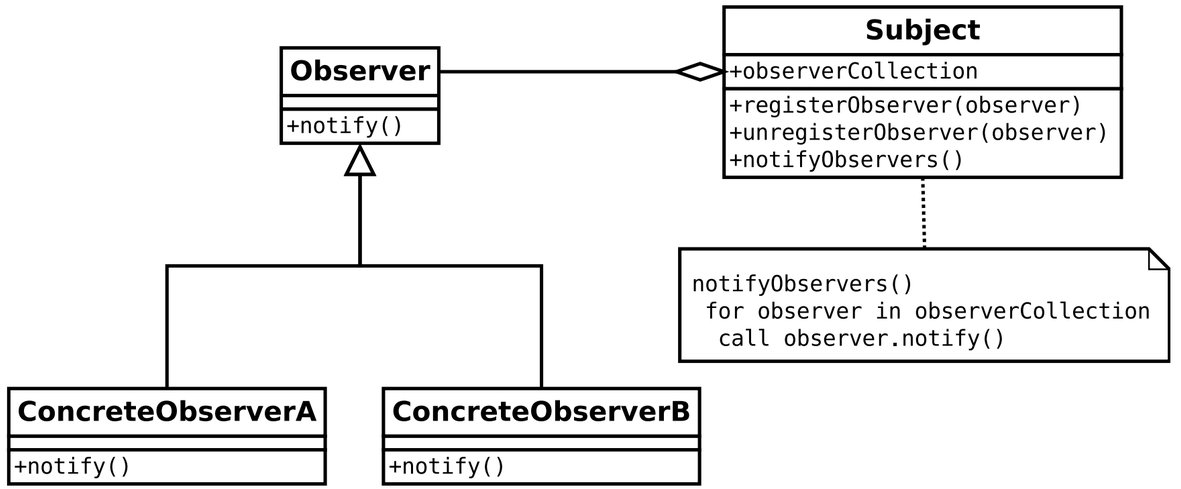
    public int getStock() {
        return stock;
    }
}
```

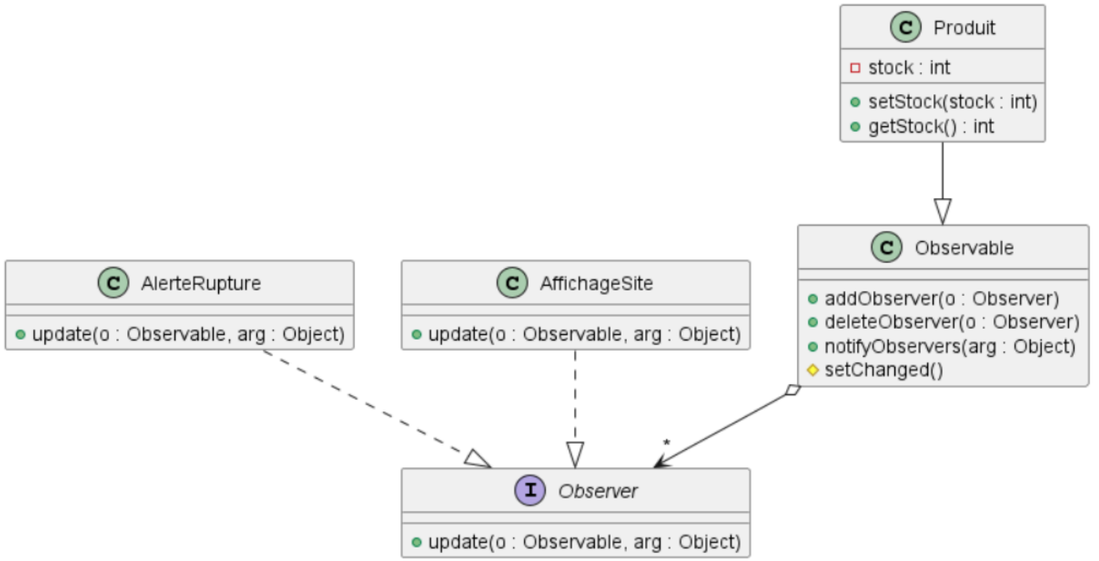
```java
public class Main {
    public static void main(String[] args) {
        Produit produit = new Produit();
        produit.addObserver(new AlerteRupture());
        produit.addObserver(new AffichageSite());

        produit.setStock(5);
        produit.setStock(0);
    }
}
```

```java
public class AffichageSite implements Observer {
    public void update(Observable o, Object arg) {
        System.out.println("Stock affiché sur le site : " + arg);
    }
}
```

```java
public class AlerteRupture implements Observer {
    public void update(Observable o, Object arg) {
        int stock = (int) arg;
        if (stock == 0) {
            System.out.println("Rupture de stock !");
        }
    }
}
```


Observer
+notify()
Subject
+observerCollection
+registerObserver(observer)
+unregisterObserver(observer)
+notifyObservers()
notifyObservers()
for observer in observerCollection
call observer.notify()
ConcreteObserverA
+notify()
ConcreteObserverB
+notify()


Produit
stock : int
setStock(stock : int)
getStock() : int
Observable
addObserver(o : Observer)
deleteObserver(o : Observer)
notifyObservers(arg : Object)
setChanged()
AlerteRupture
update(o : Observable, arg : Object)
AffichageSite
update(o : Observable, arg : Object)
Observer
update(o : Observable, arg : Object)
*

IAgraphie & Source

Julian : Création visuelle des diagramme de classe et de séquence

Eliot : idée d’exemple de la boutique

Hatim: Sonnet 5.5 Claude : Aide pour la documentation , Aide pour generer un *.puml pour generer les diagram UML

Thomas :

Source :

- https://gotan.io/fr/blog/design-patterns-gof/
- https:/refactoring.guru/fr/design-patterns/observer
- https://fr.wikipedia.org/wiki/Observateur_(patron_de_conception)#

Merci pour votre attention

